

Software Testing

Property-based Testing

João Saraiva ★, Marcos Viera ★★

2025/2026

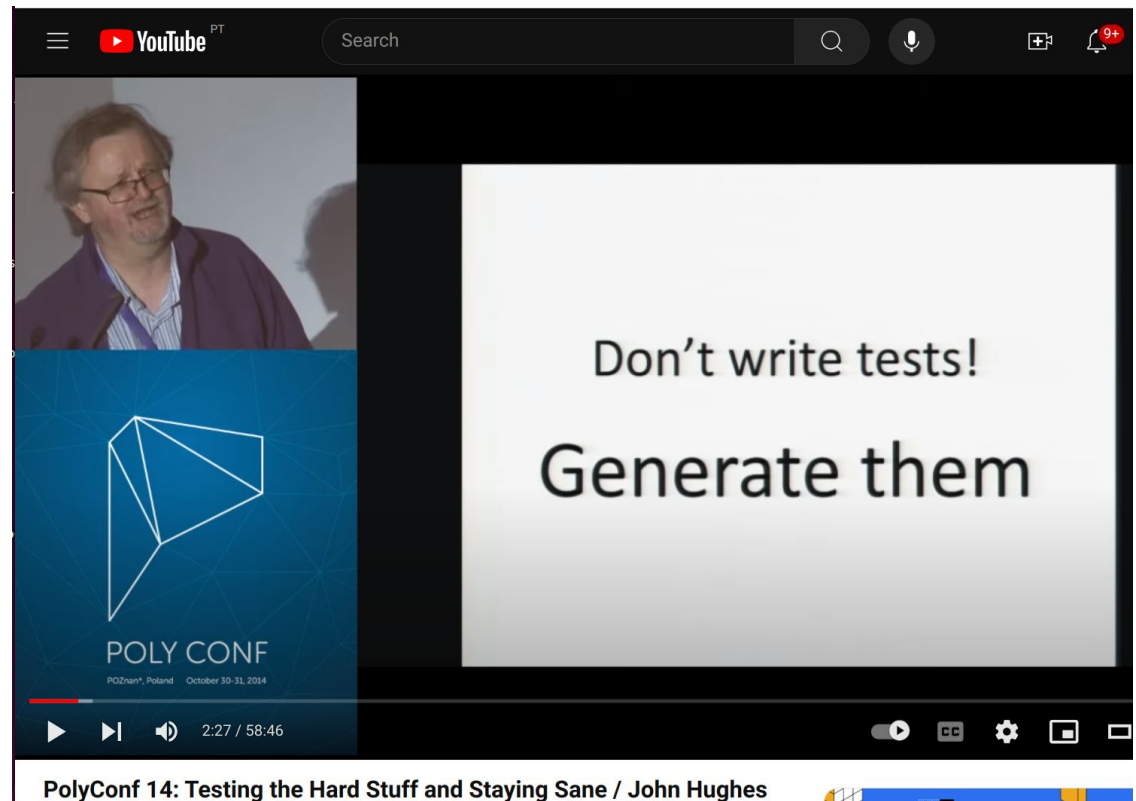
★*Departamento de Informática*
Universidade do Minho, Portugal

★★*Universidade de Montevideo*
Uruguai

Don't write tests! Generate them

John Hughes:

- ▶ One of the principal designers of the **Haskell** programming language,
- ▶ the creator of **QuickCheck**, a foundational framework in the field of **property-based testing**.



[John Hughes - Don't Write Tests!](#)

HUnit Testing

Consider the following representation of strings, where each character is paired with its position:

```
type MyString = [ (Char,Int) ]
```

The string "**Braga**" could be represented as follows:

```
braga :: MyString  
braga=[ ('a',3) , ('r',2) , ('g',4) , ('B',1) , ('a',5) ]
```

or as:

```
braga'=[ ('B',1) , ('r',2) , ('a',3) , ('g',4) , ('a',5)  
        ]
```

HUnit Testing

To convert a value of this representation into a standard string:

```
toString :: MyString -> String  
toString s = map fst  (sortOn snd s)
```

The inverse transformation:

```
fromString :: String -> MyString  
fromString s = zip s [1..]
```

HUnit Testing

We define HUnit test cases to test function toString:

```
testToEmpty      = []      ~?=  toString []  
testToSingleton  = "a"     ~?=  toString [('a',1)]  
testToMultiple   = "Braga" ~?=  toString braga
```

We define (similar) test cases to test function fromString:

```
testFromEmpty      = []      ~?=  fromString []  
testFromSingleton  = [('a',1)] ~?=  fromString "a"  
testFromMultiple   =  
    braga'          ~?=  fromString "braga"
```

HUnit Testing

we define the test suite for these two functions by enumerate the list of test cases and we use `runTestTT` to execute them:

```
*ghci> runTestTT $ TestList [testToEmpty,  
testToSingleton,testToMultiple,testFromEmpty  
,testFromSingleton,testFromMultiple]  
Cases: 6   Tried: 6   Errors: 0   Failures: 0  
Counts {cases = 6, tried = 6, errors = 0,  
failures = 0}
```

HUnit Testing: tedious and complex!

When multiple features interact, it is necessary to design test cases that validate their composition rather than merely their individual behavior.

```
testFromToEmpty =  
    [] ~?= (toString . fromString) []  
testFromToSingleton =  
    "a" ~?= (toString . fromString) "a"  
testFromToMultiple =  
    "Braga" ~?= (toString . fromString) "Braga"
```

HUnit Testing: tedious and complex!

Obviously, we need to test the other form of composing these functions (`fromString . toString`)

```
testToFromEmpty      = ...  
testToFromSingleton = ...  
testToFromMultiple   = ...
```

Consider that we need to add a new functionality to our string: the concatenation of two strings.

```
concat :: MyString -> MyString -> MyString  
concat s1 s2 = ...
```

Exercise: Define this function, and test it.

HUnit Testing: tedious and complex!

Would it not be nice if we could:

- just write down how the program should behave
- and have the test cases be generated automatically?

That is precisely the approach of **property-based testing**.

Automated Testing

**QuickChec
k**

Automated Testing - QuickCheck

QuickCheck is an excellent Haskell library that allows expressing **properties about programs** and testing them with **randomly-generated values**.

Automated Testing - QuickCheck

Import the **library** QuickCheck.

```
import Test.QuickCheck
```

Given a program que want to **test**.

```
reverse []      = []  
reverse (x:xs) = reverse xs ++ [x]
```

Define a **property** as Haskell function that returns a boolean.

```
prop_RevApp :: [Int] -> [Int] -> Bool  
prop_RevApp xs ys = reverse (xs++ys) ==  
                      reverse ys ++ reverse xs
```

Apply the function **quickCheck** to test the property.

```
*Main> quickCheck prop_RevApp  
+++ OK, passed 100 tests.
```

QuickCheck - Generators

QuickCheck provides the **Arbitrary type class** for random generation and shrinking values:

```
class Arbitrary a where
    arbitrary :: Gen a
    shrink    :: a -> [a]
```

Function **shrink** returns a list of possible immediate shrinks of a given value:

```
*Main> shrink 2
[0,1]
```

```
*Main> shrink [1,2]
[[], [2], [1], [0,2], [1,0], [1,1]]
```

QuickCheck - Properties

A property is an instance of the class Testable:

```
quickCheck :: Testable prop => prop -> IO ()
```

```
class Testable prop where  
    property :: prop -> Property
```

With the provided instances:

```
instance Testable Bool where ...
```

```
instance (Arbitrary a, Show a, Testable prop)  
=> Testable (a -> prop) where ...
```

QuickCheck - Testing

Let us consider a user defined function **addL** as follows:

```
addL :: Num a => [a] -> a
addL []          = 1
addL (h : t)    = h + addL t
```

We can define a properties to test the function:

```
prop_addLRev :: [Int] -> Bool
prop_addLRev l = addL (reverse l) == addL l
```

```
prop_addLSing :: Int -> Bool
prop_addLSing i = addL [i] == i
```

```
prop_addLSum :: [Int] -> Bool
prop_addLSum l = sum l == addL l
```

QuickCheck - Testing

Let us now consider the following implementation of **sort**:

```
import Data.Set

setSort :: Ord a => [a] -> [a]
setSort = toList . fromList
```

. The resulting list is sorted:

```
prop_sortSorted :: [Int] -> Bool
prop_sortSorted xs = sorted (setSort xs)
```

```
*Main> quickCheck prop_sortSorted
+++ OK, passed 100 tests.
```


QuickCheck - Testing

Let us now consider the following implementation of **sort**:

```
import Data.Set

setSort :: Ord a => [a] -> [a]
setSort = toList . fromList
```

- The resulting list preserves the elements of the original:

```
prop_sortElems :: [Int] -> Bool
prop_sortElems l = and (map (\x-> elem x sl) l)
    where sl = setSort l
```

```
*Main> quickCheck prop_sortElems
+++ OK, passed 100 tests.
```

QuickCheck - Testing

Let us now consider the following implementation of **sort**:

```
import Data.Set

setSort :: Ord a => [a] -> [a]
setSort = toList . fromList
```

- The resulting list preserves the length of the original:

```
prop_sortLength :: [Int] -> Bool
prop_sortLength xs = length xs == length rs
  where rs = setSort xs
```

```
*Main> quickCheck prop_sortLength
*** Failed! Falsified (after 5 tests and 1
shrink):
[-3,-3]
```

QuickCheck – Verbose Testing

We have a verbose version of quickCheck:

```
*Main> verboseCheck prop_sortLength
```

```
Passed: []  
Passed: [1]  
Passed: [2]  
Passed: []  
Passed: [2,3,-3]  
Failed: [3,4,4]  
Passed: []  
Failed: [4,4]  
Passed: []  
Passed: [4]  
Passed: [4]
```

```
Passed: [0,4]  
Passed: [2,4]  
Passed: [3,4]  
Passed: [4,0]  
Passed: [4,2]  
Passed: [4,3]  
  
*** Failed! Falsified  
(after 6 tests and 1  
shrink):  
[4,4]
```

QuickCheck – Conditional Properties

Now consider the following implementation of **replicate**:

```
replicate :: Int -> a -> [a]
replicate 0 _ = []
replicate 1 x = [x]
replicate n x = x : replicate (n-2) x
```

. If we define the following property:

```
prop_RepRev :: Eq a => Int -> a -> Bool
prop_RepRev i a = replicate i a ==
                    reverse (replicate i a)
```

```
*Main> quickCheck prop_RepRev
...
```

It doesn't terminate!

QuickCheck – Conditional Properties

We need to express that this property is only tested when its first argument is positive.

QuickCheck provides **conditional properties** to restrict the values the property is tested.

```
prop_RepRev :: Eq a => Int -> a -> Property
prop_RepRev i a = i > 0 ==>
    replicate i a == reverse (replicate i a)
```

```
*Main> quickCheck prop_RepRev
+++ OK, passed 100 tests; 117 discarded.
```

QuickCheck – Conditional Properties

```
fib :: Integer -> Integer
fib 0 = 1
fib 1 = 1
fib n = fib (n-1) + fib (n-2)

ffib = snd . fib2                -- fast fib
  where fib2 0 = (1,1)
        fib2 1 = (1,1)
        fib2 n = let (nM2,nM1) = fib2 (n-1)
                   in  (nM1 , nM2 + nM1)
```

We can make a property fail if it doesn't finish within a given number of microseconds.

```
prop_Fib :: Integer -> Property
prop_Fib n = n >= 0 ==> within (10000000)
    (ffib n == fib n)
```

QuickCheck – Conditional Properties

```
insert :: Ord a => a -> [a] -> [a]
insert x [] = [x]
insert x (y:ys) | x < y      = x:y:ys
                  | otherwise = y:insert x ys
```

```
prop_InsOrd :: Int -> [Int] -> Property
prop_InsOrd x xs = sorted xs
                  ==> sorted (insert x xs)
```

```
*Main> quickCheck prop_InsOrd
*** Gave up! Passed only 79 tests; 1000
discarded tests.
```

QuickCheck – Showing Test Cases

Info

Using `collect` we can obtain information of the test cases:

```
prop_InsOrd :: Int -> [Int] -> Property
prop_InsOrd x xs = collect (length xs) $
    sorted xs ==> sorted (insert x xs)
```

```
*Main> quickCheck prop_InsOrd
*** Gave up! Passed only 79 tests; 1000
discarded tests:
37% 0
29% 1
29% 2
 5% 3
```

Precondition is too strong!

QuickCheck – Explicit Generators

We can define a type (and instance of Arbitrary) for sorted lists:

```
data Sorted a = Sorted [a]

prop_InsOrd :: Int -> Sorted Int -> Property
prop_InsOrd x (Sorted xs) = sorted (insert x xs)
```

Or we can use an **explicit generator**:

```
prop_InsOrd :: Int -> Property
prop_InsOrd x = forAll orderedList $
    \xs -> sorted (insert x xs)
```

```
forAll :: (Show a, Testable prop)
    => Gen a -> (a -> prop) -> Property
```

QuickCheck – Choosing the instance

Finally, suppose we have the following property:

```
prop_Sorted xs = sorted xs
```

```
*Main> quickCheck prop_Sorted  
+++ OK, passed 100 tests.
```

```
prop_Sorted xs = collect xs $ sorted xs
```

```
*Main> quickCheck prop_Sorted  
+++ OK, passed 100 tests:  
6% [(), (), (), (), (), (), (), (), (), (), ()]  
6% [()]  
6% []  
...
```

Types are important!

Properties to Test?

The key challenge in making effective use of QuickCheck lies in coming up with good properties to test.

Roundtrip Properties: When one function is an inverse to another function, we can create a property to test for that.

Example 1: We can test that reversing a list is its own inverse:

```
prop_rev_rev :: [Int] -> Bool
prop_rev_rev l = reverse (reverse l) == l
```

Properties to Test?

Roundtrip Properties: When one function is an inverse to another function, we can create a property to test for that.

Example 3: We can test that inserting an element in a list and then deleting it again, results in the same list

```
prop_ins_del :: [Int] -> Int -> Bool  
prop_ins_del l x = delete x (insert x l) == l
```

where `insert` and `delete` are defined in `Data.List`

Any property of the form $f (g (\dots (h x))) == x$ is called a roundtrip property.

Properties to Test?

Equivalent Implementations: When we have two functions that should be functionally equivalent but have a different implementation, we can test that this is indeed the case.

Example 1: The prelude function `sum` can be defined as a `foldr` and we can define a property to test it.

```
sum'  :: [Int] -> Int
sum'  = foldr (+) 0
```

```
prop_sum :: [Int] -> Bool
prop_sum l = sum l == sum' l
```

Properties to Test?

Equivalent Implementations: When we have two functions that should be functionally equivalent but have a different implementation, we can test that this is indeed the case.

Example 1: The `quickSort` and `bubbleSort` functions have the same behaviour.

```
quickSort, bubbleSort :: Ord a => [a] -> [a]  
...
```

```
prop_sort :: [Int] -> Bool
```

```
prop_sort l = quickSort l == bubbleSort l
```

Properties to Test?

Invariants: An invariant of a function is a statement that, if it holds before calling the function, then it holds after the function returns.

Example 1: The length of a list is preserved by sorting and reversing a list.

```
prop_sort_len :: [Int] -> Bool
prop_sort_len l = length l == length (sort l)

prop_rev_len :: [Int] -> Bool
prop_rev_len l =
    length l == length (reverse l)
```

Properties to Test?

Algebraic Laws: When we have a function that implements a certain idea from algebra, we can use the algebraic laws as inspiration to define properties.

Example 1: Consider the following expressions and its simple evaluator.

```
data Exp = Add Exp Exp
         | Mul Exp Exp
         | Neg Exp
         | Const Int

eval :: Exp -> Int
eval (Const i) = i
eval (Neg e)    = - (eval e)
eval (Add e d)  = eval e + eval d
eval (Mul e d)  = eval e * eval d
```


Properties to Test?

Algebraic Laws: We can define properties to test that the addition in the Exp evaluator is commutative, has a neutral elements, is associative, etc.

```
prop_add_neutral :: Exp -> Bool
prop_add_neutral e =
    eval e == eval (Add e (Const 0))
    && eval e == eval (Add (Const 0) e)

prop_add_comutative :: Exp -> Exp -> Bool
prop_add_comutative e1 e2 =
    eval (Add e1 e2) == eval (Add e2 e1)

...
```

Properties to Test?

Algebraic Laws - Idempotent: A function is idempotent when applying it twice produces the same result as applying them just once.

Example 1: Sorting functions are idempotent.

```
prop_sort_idempotent :: [Int] -> Bool
prop_sort_idempotent l =
    sort (sort l) == sort l
```

Example 2: The nub function that removes duplicate elements from a list is idempotent

```
prop_nub_idempotent :: [Int] -> Bool
prop_nub_idempotent l = nub (nub l) == nub l
```

Properties to Test?

Algebraic Laws - Idempotent.

Example 1: Write a property to test whether the following `opt` function is idempotent.

```
opt :: Exp -> Exp
opt (Add (Const 0) e) = opt e
opt (Add e (Const 0)) = opt e
opt (Add (Const a) (Const b)) = Const (a + b)
opt (Neg (Const a))          = Const (-a)
opt (Mul e1 e2)              = Mul (opt e1) (opt e2)
opt (Add e1 e2)              = Add (opt e1) (opt e2)
opt e                        = e
```

Properties to Test?

Algebraic Laws - Idempotent .

```
prop_opt_idempotent :: Exp -> Bool
prop_opt_idempotent e = opt (opt e) == opt e
```

```
*Main> quickCheck prop_opt_idempotent
*** Failed! Falsified (after 2 tests):
Add (Neg (Const 0)) (Const (-1))
```

opt is not idempotent

Properties to Test?

This result also shows that our simple definition of `opt` does not fully apply the desired optimizations.

```
--      (neg 4  +  4) + 5      evals to 5
e = Add (Add (Neg (Const 4)) (Const 4)) (Const 5)
```

```
*Main> opt e
Add (Add (Const (-4)) (Const 4)) (Const 5)
ghci> opt $ opt e
Add (Const 0) (Const 5)
ghci> opt $ opt $ opt e
Const 5
ghci> opt $ opt $ opt $ opt e
Const 5
```

A fix point is reached

Properties to Test?

Thus, we need to express `opt` using fixpoint computation

```
fix :: Eq a => (a -> a) -> a -> a
fix f x | x == x'      = x
        | otherwise    = fix f x'
    where x' = f x
```

```
opt' :: Exp -> Exp
opt' = fix opt
```

```
prop_opt_idempotent' :: Exp -> Bool
prop_opt_idempotent' e = opt' (opt' e) == opt' e
```

```
*Main> quickCheck prop_opt_idempotent'
+++ OK, passed 100 tests.
```

Shrinking...

When testing if `opt` is idempotent the property failed and we got a small/intuitive **Expr** counter example:

```
*Main> quickCheck prop_opt_idempotent  
*** Failed! Falsified (after 2 tests):  
Add (Neg (Const 0)) (Const (-1))
```

Values of type **Expr** are produced by our defined `genExpr` generator, which is able to produce **large expressions**.

How was `quickCheck` able to produce such a simple and readable counter example?!

Shrinking...

By “shrinking” the test case/values it generates.

Otherwise....:

```
*Main> quickCheck prop_opt_idempotent
*** Failed! Falsified (after 12 tests):
Add (Add (Add (Mul (Neg (Const (-3))) (Neg
(Neg (Neg (Const (-3)))))) (Neg (Mul (Const
(-9)) (Const 8)))) (Mul (Neg (Mul (Neg (Neg
(Const 5))) (Const 4))) (Mul (Neg (Neg (Const
2))) (Neg (Neg (Neg (Const 8))))))) (Add
(Const (-8)) (Neg (Const (-5))))
```


Shrinking

Shrinking is the process by which QuickCheck tries to produce human readable examples when it finds a failure - it takes a complex example and turns it into a simpler one.

```
data BTree a = Node a (BTree a) (BTree a)
              | Leaf
```

Heuristic to shrink a binary tree:

in a branching node:

- *a) replace it by a leaf;*
- *b) replace it by the left or right subtrees;*
- *c) shrink the left or right subtrees;*
- *d) shrink the value.*

no further shrinking when reaching a leaf.

Shrinking

Thus, we need to express `opt` using fixpoint computation

```
shrinkBTree :: Arbitrary a
             => BTree a -> [BTree a]
shrinkBTree (Node v l r) =
    [Leaf, l, r]                -- a) b)
  ++ [Node v l' r | l' <- shrinkTree l]    -- c)
  ++ [Node v l r' | r' <- shrinkTree r]    -- c)
  ++ [Node v' l r | v' <- shrink v]        -- d)
shrinkBTree Leaf = []            -- no shrinking
                                -- for leaves
```

Shrinking

Thus, we can shrink expressions as follows

```
shrinkExp :: Expr -> [Expr]
shrinkExp (Add l r) =
    [l, r]
    ++ [Add l' r | l' <- shrinkExp l]
    ++ [Add l r' | r' <- shrinkExp r]
shrinkExp (Mul l r) =
    [l, r]
    ++ [Mul l' r | l' <- shrinkExp l]
    ++ [Mul l r' | r' <- shrinkExp r]
shrinkExp (Neg e)    = [e] ++
    [Neg e' | e' <- shrinkExp e]
shrinkExp (Const i) = []
```

Shrinking...

Hum... We are no “shrinking” the constants (Int)...

```
*Main> quickCheck prop_opt_idempotent  
*** Failed! Falsified (after 4 tests and 1  
shrink):  
Add (Const (-3)) (Neg (Const (-1)))
```

Shrinking

Thus, we can shrink expressions as follows

```
shrinkExp :: Expr -> [Expr]
shrinkExp (Add l r) =
    [l, r]
    ++ [Add l' r | l' <- shrinkExp l]
    ++ [Add l r' | r' <- shrinkExp r]
shrinkExp (Mul l r) =
    [l, r]
    ++ [Mul l' r | l' <- shrinkExp l]
    ++ [Mul l r' | r' <- shrinkExp r]
shrinkExp (Neg e)    = [e] ++
    [Neg e' | e' <- shrinkExp e]
shrinkExp (Const i) = [Const i'
    | i' <- shrink i ]
```

Shrinking...

.

```
*Main> quickCheck prop_opt_idempotent  
*** Failed! Falsified (after 4 tests and 6  
shrinks):  
Add (Add (Const 0) (Const 0)) (Const 1)
```

Shrinking

And the instance of `Arbitrary` defines function `shrink` now.

```
instance Arbitrary Expr where
  arbitrary = sized genExpr
  shrink    = shrinkExp
```

Obrigado!